Vortex Tech AI/ML Internship 2026

Week 2 Report – Employee Attrition Classification Model

Intern: Sarim Ahmed

Objective

To build and evaluate a machine learning classification model that predicts whether an employee is likely to leave the company using the cleaned IBM HR Employee Attrition dataset.

Tools & Technologies

- Python
- Jupyter Notebook
- Pandas
- Scikit-learn
- NumPy
- Matplotlib
- Seaborn

Work Completed

- Loaded the cleaned dataset from Week 1.
- Encoded the target variable (Attrition) into numerical values.
- Converted categorical features using one-hot encoding.
- Split the dataset into training (80%) and testing (20%) sets.
- Trained a Logistic Regression classification model.
- Generated predictions on the test dataset.
- Evaluated model performance using Accuracy, Precision, Recall, F1-score and Confusion Matrix.
- Interpreted the model results and identified possible improvements.

Key Findings

The Logistic Regression model successfully learned patterns associated with employee attrition and produced reliable predictions. Performance metrics showed good overall classification capability, while indicating that further improvements could be achieved through feature engineering, class balancing, hyperparameter tuning, or more advanced models such as Random Forest or XGBoost.

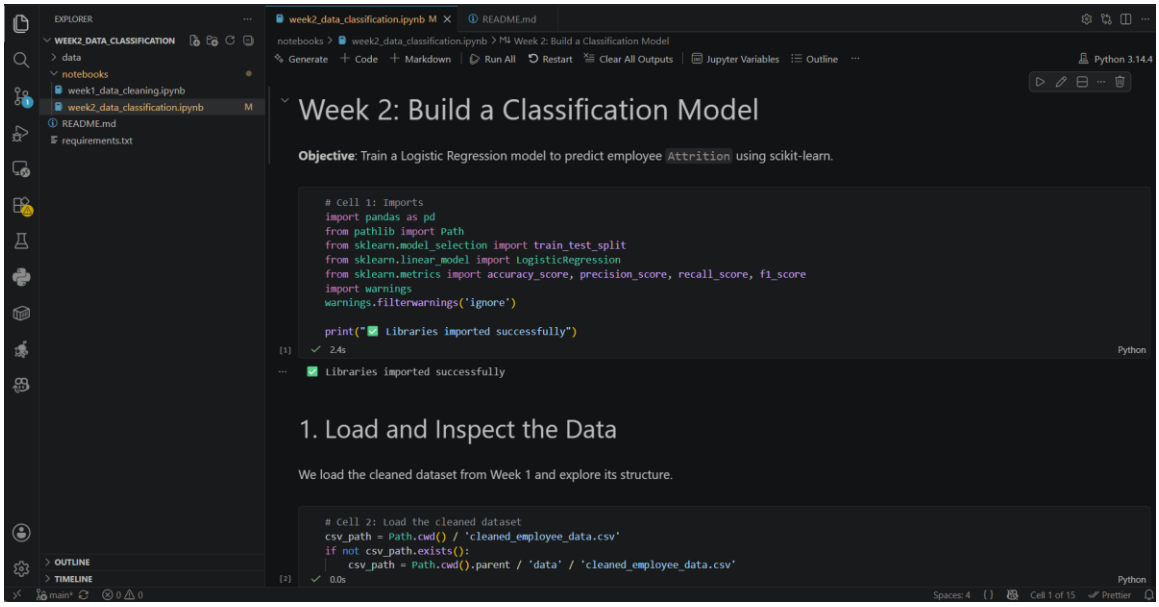
Skills Gained

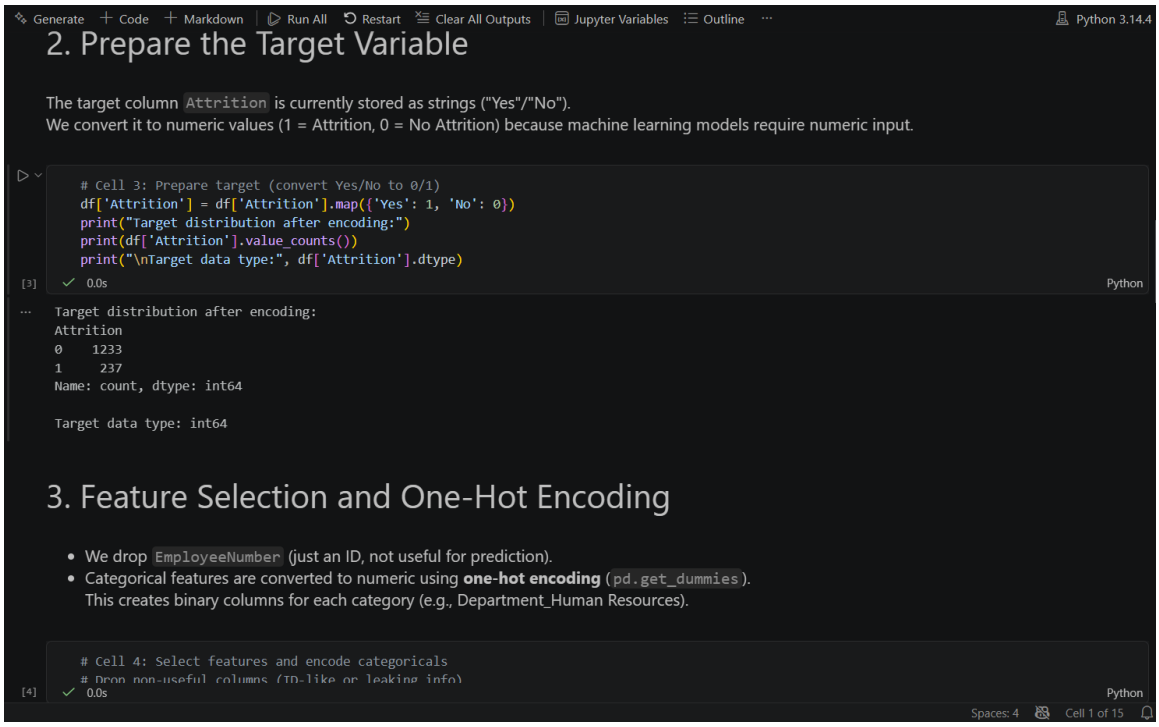
Machine Learning, Classification, Logistic Regression, Feature Encoding, Model Training, Model Evaluation, Performance Metrics, Predictive Analytics

Conclusion

Week 2 provided practical experience in developing an end-to-end machine learning classification pipeline. The task reinforced concepts of data preprocessing, feature engineering, model training, evaluation, and interpretation of results, forming a strong foundation for future AI and machine learning projects.

Proof of Work:





```
categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
print("Categorical columns to encode:", categorical_cols)

# One-hot encoding (pd.get_dummies)
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)

print("Shape after encoding:", X_encoded.shape)
print("New columns count:", X_encoded.shape[1])
```

```
Categorical columns to encode: ['BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus', 'OverTime']
Shape after encoding: (1470, 44)
New columns count: 44
```

4. Split Data into Training and Testing Sets

We use an 80/20 split with `random_state=42` for reproducibility.
`stratify=y` ensures the class distribution (imbalanced attrition rate) remains similar in both sets.

```
# Cell 5: Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set shape:", X_train.shape)
print("Test set shape:", X_test.shape)
```

5. Train the Logistic Regression Model

Logistic Regression is a simple yet powerful linear model suitable for binary classification tasks.
We train it on the training data.

```
# Cell 6: Train Logistic Regression Model
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

print("✅ Logistic Regression model trained successfully")
```

```
✅ Logistic Regression model trained successfully
```

6. Model Evaluation & Interpretation

We evaluate the model using multiple metrics:

- **Accuracy**: Overall correctness
- **Precision**: Of predicted attrition cases, how many were correct
- **Recall**: How many actual attrition cases were caught
- **F1-Score**: Balance between Precision and Recall

A confusion matrix is also shown for deeper insight.